



Université de Nouakchott

Faculté des Sciences et Techniques (FST)

Département de Mathématiques et Informatique

Rapport de Projet de Fin d'Études

Pour l'Obtention de la Licence Académique en Mathématiques et Informatique

Thème :

**Conception d'un Agent Conversationnel Intelligent
pour les Services de l'Université de Nouakchott**

Présenté par les étudiants : **Encadrant Pédagogique :**

Mohamed Salim Aly

Beide Bouh Cheikhna

Ahmed Yaghoub Bedi Be

Année Universitaire : 2025 - 2026

Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Je remercie en premier lieu **Allah, le Tout-Puissant**, qui m'a accordé la force, la patience et la volonté nécessaires pour mener à bien ce projet.

Mes sincères remerciements vont à **mon encadrant**, dont les conseils éclairés, la disponibilité constante et les orientations pertinentes ont été d'une valeur inestimable tout au long de ce projet. Sa rigueur scientifique et son soutien moral ont été des piliers fondamentaux dans l'accomplissement de ce travail.

Je remercie également l'ensemble du corps **enseignant de la Faculté des Sciences et Techniques** de l'Université de Nouakchott, pour la formation de qualité dispensée tout au long de mon cursus universitaire.

Ma gratitude s'adresse aussi aux membres du **jury** qui ont accepté d'évaluer ce travail et de consacrer leur temps précieux à sa lecture et à son appréciation.

Enfin, j'exprime toute ma reconnaissance à **ma famille**, en particulier mes parents, pour leur soutien indéfectible, leurs encouragements et leurs sacrifices. Je remercie également mes **amis et camarades de promotion** pour l'ambiance fraternelle et les échanges enrichissants.

À tous, je dis merci.

Dédicace

À mes chers parents,

*qui ont tout sacrifié pour que je puisse aller au bout de mes rêves,
dont le soutien, l'amour et les prières
ont éclairé chaque étape de mon parcours.*

À mes frères et sœurs,

pour leur amour et leurs encouragements constants.

À mes enseignants,

qui ont planté en moi les graines du savoir.

À mes amis,

compagnons de route dans cette belle aventure universitaire.

Résumé

Ce projet de fin d'études présente la conception et l'implémentation d'un chatbot universitaire bilingue (arabe-français) destiné à l'Université de Nouakchott (UN). Le système vise à fournir un accès instantané aux informations académiques de l'université, couvrant ses cinq facultés : FST, FLSH, FEG, FSJP et FMPOS.

L'architecture repose sur quatre composants communiquant via des API REST : un moteur conversationnel **Rasa 3.6** pour la compréhension du langage naturel (NLU) et la gestion du dialogue, un serveur d'actions **Python/Rasa SDK**, un backend **Flask** avec persistance **SQLite**, et une interface **React.js**. La solution est conteneurisée avec **Docker Compose**.

Les tests ont montré un F1-Score NLU global de 86,7% et un temps de réponse médian inférieur à 750 ms. Le système inclut un tableau de bord d'administration pour la gestion des questions non répondues.

Mots-clés : Chatbot, Rasa, NLU, Intelligence Artificielle, TALN, Flask, React, Docker, Bilingue, Arabe, Français.

Abstract

This final year project presents the design and implementation of a bilingual (Arabic-French) university chatbot for the University of Nouakchott (UN). The system provides instant access to academic information across five faculties : FST, FLSH, FEG, FSJP, and FMPOS.

The architecture comprises four components communicating via REST APIs : a **Rasa 3.6** conversational engine for NLU and dialogue management, a **Python/Rasa SDK** action server, a **Flask** backend with **SQLite** persistence, and a **React.js** frontend. The solution is containerized with **Docker Compose**.

Tests showed a global NLU F1-Score of 86.7% and a median response time below 750 ms. An admin dashboard enables management of unanswered questions.

Keywords : Chatbot, Rasa, NLU, Artificial Intelligence, NLP, Flask, React, Docker, Bilingual, Arabic, French.

Table des matières

Remerciements	i
Dédicace	ii
Résumé	iii
Abstract	iv
Liste des figures	viii
Liste des tableaux	ix
Liste des abréviations	x
Introduction Générale	1
1 Contexte et Cadre du Projet	2
1.1 Introduction	2
1.2 Contexte général et enjeux du numérique universitaire	2
1.3 Présentation de l'Université de Nouakchott	2
1.4 Présentation de la Faculté des Sciences et Techniques (FST)	3
1.5 Analyse de l'existant	4
1.6 Problématique	4
1.7 Objectifs du projet	4
1.8 Cahier des charges	5
1.9 Contraintes du projet	5
1.10 Conclusion	5
2 État de l'Art	6
2.1 Introduction	6
2.2 Intelligence Artificielle et Traitement Automatique du Langage Naturel	6
2.3 Les systèmes conversationnels (Chatbots)	7
2.4 Évolution des approches conversationnelles	7
2.4.1 Approche basée sur des règles	7
2.4.2 Approche basée sur l'apprentissage automatique	7
2.4.3 Approche basée sur les modèles Transformers	7
2.5 Présentation des frameworks de développement de chatbots	7

2.5.1	Rasa	7
2.5.2	Dialogflow	8
2.5.3	Microsoft Bot Framework	8
2.6	Étude comparative des solutions existantes	8
2.7	Cas d'utilisation des chatbots dans les universités	8
2.8	Choix technologique	8
2.9	Conclusion	9
3	Analyse et Conception	10
3.1	Introduction	10
3.2	Identification des acteurs	10
3.3	Analyse des besoins	11
3.3.1	Besoins fonctionnels	11
3.3.2	Besoins non fonctionnels	11
3.4	Spécification des cas d'utilisation	12
3.5	Diagrammes de cas d'utilisation	13
3.6	Architecture générale du système	13
3.7	Conception du système conversationnel	14
3.8	Diagrammes UML	15
3.8.1	Diagramme de séquence	15
3.8.2	Diagramme de classes	16
3.9	Modèle de données	16
3.10	Conclusion	16
4	Réalisation et Implémentation	17
4.1	Introduction	17
4.2	Environnement de développement	17
4.3	Technologies utilisées	18
4.4	Structure du projet	18
4.5	Implémentation du Backend Flask	18
4.6	Implémentation du moteur conversationnel Rasa	19
4.7	Développement des actions personnalisées	20
4.8	Implémentation du Frontend React	21
4.9	Interface utilisateur principale	23
4.10	Base de données SQLite	25
4.11	Déploiement avec Docker	25
4.12	Hébergement et mise en production	25
4.13	Sécurité et protection des données	26
4.14	Conclusion	26

5 Tests et Validation	27
5.1 Introduction	27
5.2 Stratégie de test	27
5.3 Tests fonctionnels	28
5.4 Tests NLU	28
5.5 Tests d'intégration	29
5.6 Tests de performance	30
5.7 Évaluation des résultats	30
5.8 Analyse critique	30
5.9 Conclusion	31
Conclusion Générale et Perspectives	32
Bibliographie	34

Table des figures

3.1	Diagramme de cas d'utilisation global	13
3.2	Architecture générale du système Chatbot UN	13
3.3	Diagramme de séquence — interaction utilisateur typique	15
3.4	Diagramme de classes simplifié	16
4.1	Tableau de bord administrateur en production — Mode clair (gauche) et mode sombre (droite) montrant : 12 questions sans réponse, 0 questions répondues, 4 utilisateurs actifs, répartition des langues (50/50 arabe/français), classement des facultés (FST 230, FLSH 185, FMPOS 140, FSJP 105, FEG 70), liste des utilisateurs actifs, et tableau détaillé des questions avec dates et actions disponibles. L'interface supporte complètement le bilinguisme arabe/français dans tous les menus et étiquettes.	22
4.2	Chatbot Universitaire en production — Mode clair (gauche) et mode sombre (droite) montrant l'interface principale avec le logo UN, l'indicateur de connexion "2 / Live", les quatre boutons de suggestions rapides (LMD, Résultats, Bourses, Facultés), le champ d'entrée bilingue et le support complet arabe/français. L'interface responsive s'adapte à tous les appareils (mobile, tablette, desktop).	23
4.3	Exemple de conversation en production — L'image montre une interaction réelle où l'utilisateur pose une question hors domaine en français ("Qui a gagné la coupe du monde 2026?"), le chatbot reconnaît correctement que la question dépasse son domaine de compétences et propose une réponse appropriée de fallback, redirigeant l'utilisateur vers les services universitaires. Les icônes de réaction (copie, like, commentaire) permettent le feedback utilisateur. Chaque message affiche l'horodatage précis (17 :57, 17 :58).	24

Liste des tableaux

1.1	Chiffres clés de l’Université de Nouakchott	3
1.2	Filières de la FST par niveau	3
2.1	Comparaison des frameworks de développement de chatbots	8
3.1	Besoins fonctionnels du système	11
3.2	Cas d’utilisation UC01 — Dialoguer avec le chatbot	12
3.3	Cas d’utilisation UC02 — Tableau de bord administrateur	12
4.1	Environnement de développement	17
4.2	Technologies et bibliothèques utilisées	18
4.3	Volume des données NLU par faculté	20
5.1	Résultats des tests fonctionnels	28
5.2	Résultats NLU par catégorie d’intentions	29
5.3	Résultats des tests d’intégration	29
5.4	Temps de réponse par type de requête	30
5.5	Synthèse de l’évaluation du système	30

Liste des abréviations

UN	Université de Nouakchott
FST	Faculté des Sciences et Techniques
FLSH	Faculté des Lettres et Sciences Humaines
FEG	Faculté d'Économie et de Gestion
FSJP	Faculté des Sciences Juridiques et Politiques
FMPOS	Faculté de Médecine, de Pharmacie et d'Odontostomatologie
IA	Intelligence Artificielle
TALN	Traitement Automatique du Langage Naturel
NLU	Natural Language Understanding
NLG	Natural Language Generation
ML	Machine Learning
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
CORS	Cross-Origin Resource Sharing
WAL	Write-Ahead Logging
LMD	Licence-Master-Doctorat
PFE	Projet de Fin d'Études
DIET	Dual Intent and Entity Transformer
TED	Transformer Embedding Dialogue
UML	Unified Modeling Language
SPA	Single Page Application

Introduction Générale

L'essor fulgurant de l'intelligence artificielle et des technologies numériques a profondément reconfiguré les modes d'interaction entre les individus et les institutions. Dans ce contexte de transformation digitale généralisée, les établissements d'enseignement supérieur se trouvent confrontés à un double défi : répondre à une demande croissante d'informations de la part d'un nombre toujours plus grand d'étudiants, tout en optimisant leurs ressources humaines et administratives.

L'Université de Nouakchott (UN), principale institution d'enseignement supérieur en Mauritanie, accueille des milliers d'étudiants répartis dans plusieurs facultés et départements. Cette diversité génère un flux constant de questions portant sur les programmes d'études, les modalités d'inscription, les calendriers académiques, les filières disponibles et de nombreuses autres thématiques. La gestion de ces requêtes, souvent répétitives, mobilise d'importantes ressources administratives et aboutit parfois à des délais de réponse insatisfaisants.

C'est dans ce contexte que s'inscrit le présent projet de fin d'études, qui propose la conception et l'implémentation d'un **chatbot universitaire bilingue (arabe-français)** capable de répondre automatiquement et en temps réel aux questions des utilisateurs concernant l'université et ses composantes.

Organisation du rapport

Ce rapport est structuré en cinq chapitres :

- **Chapitre 1** : Contexte, présentation de l'UN/FST, analyse de l'existant, problématique et objectifs.
- **Chapitre 2** : État de l'art sur le TALN, les chatbots et les frameworks existants.
- **Chapitre 3** : Analyse et conception — diagrammes UML, architecture et modèle de données.
- **Chapitre 4** : Réalisation et implémentation des composants du système.
- **Chapitre 5** : Tests, validation et analyse critique des résultats.

Chapitre 1

Contexte et Cadre du Projet

1.1 Introduction

Ce premier chapitre pose les bases du projet : contexte général, présentation de l'université et de la faculté, analyse de l'existant, problématique, objectifs et cahier des charges.

1.2 Contexte général et enjeux du numérique universitaire

La transformation numérique est aujourd'hui un impératif stratégique pour les universités du monde entier. Selon l'UNESCO [6], les établissements qui investissent dans les outils numériques améliorent significativement la satisfaction de leurs étudiants et l'efficacité de leurs services administratifs.

En Afrique et dans le monde arabe, cette transformation prend une dimension particulière : face à une démographie estudiantine en forte croissance et à des ressources souvent limitées, les technologies numériques représentent un levier essentiel. L'intelligence artificielle, et plus particulièrement les agents conversationnels, offrent une réponse adaptée à ces enjeux.

Les chatbots universitaires présentent plusieurs avantages majeurs :

- **Disponibilité permanente** : opérationnel 24h/24, 7j/7.
- **Scalabilité** : traitement simultané de milliers de requêtes.
- **Réduction des coûts** : automatisation des réponses aux questions récurrentes.
- **Cohérence de l'information** : réponses uniformes et à jour.

1.3 Présentation de l'Université de Nouakchott

Fondée en 1981, l'Université de Nouakchott est la plus ancienne et la plus grande université de Mauritanie. Elle dispense des formations dans le cadre du système **LMD**

(Licence-Master-Doctorat).

TABLE 1.1 – Chiffres clés de l'Université de Nouakchott

Indicateur	Valeur
Année de fondation	1981
Nombre de facultés	5
Nombre d'étudiants	Plus de 25 000
Nombre de départements	15+
Système pédagogique	LMD
Langues d'enseignement	Arabe et Français
Localisation	Nouakchott, Mauritanie

Les cinq facultés de l'UN sont : la FST, la FLSH, la FEG, la FSJP et la FMPOS.

1.4 Présentation de la Faculté des Sciences et Techniques (FST)

La FST forme des cadres scientifiques et techniques contribuant au développement socio-économique de la Mauritanie. Elle organise ses formations sur trois cycles LMD :

- **L1** : trois parcours fondamentaux — MPI (Mathématiques-Physique-Informatique), BG (Biologie-Géologie) et PC (Physique-Chimie).
- **L2/L3** : spécialisations — DA2I, MIAGE, MI, MA, EEA, PF, TSER, BMP, BOE, GEOSS, GEOMIN, CHIMIE, PEPE, etc.
- **Master et Doctorat** : formations spécialisées et recherche.

TABLE 1.2 – Filières de la FST par niveau

Niveau	Parcours / Filières	Domaine
L1	MPI	Maths, Physique, Informatique
L1	BG	Biologie, Géologie
L1	PC	Physique, Chimie
L3	DA2I, MIAGE, MI, MA	Informatique et Mathématiques
L3	EEA, PF, TSER	Physique et Électronique
L3	BMP, BOE, NS	Biologie
L3	GEOSS, GEOMIN	Géologie
L3	CHIMIE, PEPE	Chimie

1.5 Analyse de l'existant

Moyens d'information actuels

- **Site web officiel** : présente des informations générales mais n'est pas toujours mis à jour en temps réel.
- **Permanences administratives** : limitées à des horaires de bureau, générant des files d'attente.
- **Groupe de messagerie** : WhatsApp et Telegram utilisés informellement, sans garantie de fiabilité.
- **Affichage physique** : panneaux dans les couloirs, à portée géographique limitée.

Limites identifiées

1. Absence d'un système interactif disponible 24h/24.
2. Information dispersée sur plusieurs canaux non coordonnés.
3. Aucune automatisation des réponses aux questions récurrentes.
4. Surcharge du personnel administratif.
5. Absence de support bilingue structuré (arabe/français).

1.6 Problématique

Comment concevoir et implémenter un système conversationnel intelligent, bilingue (arabe-français), capable de répondre de manière autonome et précise aux questions des étudiants de l'Université de Nouakchott, en assurant disponibilité permanente et maintenance aisée ?

1.7 Objectifs du projet

Objectif principal : Développer un chatbot universitaire bilingue fonctionnel, déployable et maintenable.

Objectifs spécifiques :

1. Concevoir une architecture microservices robuste et scalable.
2. Implémenter un moteur NLU bilingue couvrant plus de 180 intentions.
3. Développer une interface utilisateur moderne et responsive.
4. Mettre en place un système de gestion des questions non répondues.
5. Conteneuriser la solution avec Docker Compose.

1.8 Cahier des charges

Besoins fonctionnels

1. Comprendre et répondre en arabe et en français.
2. Fournir des informations sur toutes les facultés de l'UN.
3. Répondre aux questions sur les filières, matières et programmes (L1, L2, L3, Master, Doctorat).
4. Gérer les questions hors domaine via un mécanisme de fallback.
5. Enregistrer les questions sans réponse dans une base de données.
6. Proposer un tableau de bord d'administration.
7. Offrir des suggestions rapides à l'utilisateur.

Besoins non fonctionnels

1. **Performance** : temps de réponse inférieur à 3 secondes.
2. **Disponibilité** : 99% grâce aux redémarrages automatiques Docker.
3. **Scalabilité** : ajout de facultés sans refonte de l'architecture.
4. **Sécurité** : tableau de bord protégé par authentification.
5. **Portabilité** : déployable sur tout environnement Docker.
6. **Maintenabilité** : code modulaire et documenté.

1.9 Contraintes du projet

- **Linguistique** : gestion simultanée de l'arabe (RTL, morphologie complexe) et du français.
- **Données** : absence de corpus préexistants — données créées manuellement.
- **Temporelle** : calendrier d'un PFE de licence.
- **Matérielle** : entraînement Rasa sur matériel standard sans GPU dédié.

1.10 Conclusion

Ce chapitre a posé les bases du projet : présentation de l'UN et de la FST, analyse des lacunes existantes et définition précise de la problématique. Le chapitre suivant dresse l'état de l'art des technologies retenues.

Chapitre 2

État de l'Art

2.1 Introduction

Ce chapitre présente une revue des technologies dans le domaine des agents conversationnels : fondements théoriques du TALN, évolution des approches, comparaison des frameworks et justification du choix technologique.

2.2 Intelligence Artificielle et Traitement Automatique du Langage Naturel

Intelligence Artificielle

L'IA est la branche de l'informatique qui vise à créer des systèmes capables d'effectuer des tâches normalement associées à l'intelligence humaine [1]. Elle englobe l'apprentissage automatique (ML) et l'apprentissage profond (DL).

Traitement Automatique du Langage Naturel (TALN)

Le TALN est un sous-domaine de l'IA traitant de l'interaction entre les ordinateurs et le langage humain. Ses principales tâches sont :

- **Compréhension du langage (NLU)** : interprétation sémantique du texte.
- **Génération du langage (NLG)** : production de texte naturel.
- **Classification d'intentions** : identification de ce que l'utilisateur veut.
- **Extraction d'entités** : identification des éléments informatifs dans une phrase.

La révolution des architectures Transformers [2] a produit des modèles comme BERT et GPT, établissant de nouveaux standards en TALN.

2.3 Les systèmes conversationnels (Chatbots)

Un chatbot est un programme conçu pour simuler une conversation humaine. On distingue :

- **Chatbots à domaine fermé (task-oriented)** : spécialisés dans un domaine précis — c'est la catégorie de notre projet.
- **Chatbots à domaine ouvert (chit-chat)** : capables de tenir une conversation générale.

2.4 Évolution des approches conversationnelles

2.4.1 Approche basée sur des règles

Les premiers systèmes, comme ELIZA [4] (1966), utilisaient des règles prédéfinies. Leur avantage est la prévisibilité, mais leur rigidité les rend inadaptés aux interactions complexes.

2.4.2 Approche basée sur l'apprentissage automatique

Les modèles statistiques (n-grammes, LSTM, GRU) ont permis de traiter la variabilité du langage humain au prix de grandes quantités de données.

2.4.3 Approche basée sur les modèles Transformers

L'architecture Transformer [2] et le mécanisme d'attention ont révolutionné le TALN. BERT et ses dérivés établissent de nouveaux standards. Le classifieur DIET de Rasa s'inspire directement de ces architectures.

2.5 Présentation des frameworks de développement de chatbots

2.5.1 Rasa

Rasa [3] est un framework open-source basé sur le ML. Il comprend :

- **Rasa NLU** : classification des intentions et extraction d'entités.
- **Rasa Core** : gestion du dialogue et sélection des actions.

Ses atouts : open-source, déploiement local, hautement configurable, pipeline multi-langue.

2.5.2 Dialogflow

Plateforme Google cloud. Inconvénients pour notre projet : dépendance à internet, coûts API, contrôle limité du modèle.

2.5.3 Microsoft Bot Framework

Puissant mais courbe d'apprentissage élevée et dépendance à Azure.

2.6 Étude comparative des solutions existantes

TABLE 2.1 – Comparaison des frameworks de développement de chatbots

Critère	Rasa	Dialogflow	Microsoft BF
Licence	Open-source	Freemium	Propriétaire
Déploiement local	✓	✗	Partiel
Support arabe	✓	✓	✓
Coût	Gratuit	Facturation	Azure
Contrôle du modèle	Total	Limité	Limité
Intégration Docker	✓	N/A	Partiel
Données hors cloud	✓	✗	✗

2.7 Cas d'utilisation des chatbots dans les universités

- **Georgia State University (USA)** : le chatbot Pounce a réduit les abandons étudiants [7].
- **Deakin University (Australie)** : IBM Watson répond à 36 000 étudiants.
- **Universités africaines** : initiatives similaires dans des contextes multilingues.

2.8 Choix technologique

Sur la base de l'étude comparative, notre choix s'est porté sur :

- **Rasa 3.6** : moteur NLU/dialogue open-source, local, multilingue.
- **Flask** : micro-framework Python pour le backend API REST.
- **React.js** : bibliothèque JavaScript pour l'interface utilisateur.
- **SQLite** : base de données légère et sans serveur.
- **Docker Compose** : orchestration et portabilité du déploiement.

2.9 Conclusion

L'état de l'art a justifié le choix de Rasa complété par un écosystème cohérent. Le chapitre suivant traduit ces choix en architecture concrète.

Chapitre 3

Analyse et Conception

3.1 Introduction

Ce chapitre couvre l'identification des acteurs, l'analyse des besoins, la spécification des cas d'utilisation et la conception architecturale et UML du système.

3.2 Identification des acteurs

1. **Étudiant / Visiteur** : utilisateur principal, interagit avec le chat sans authentification.
2. **Administrateur** : accède au tableau de bord (mot de passe requis) pour gérer les questions non répondues.
3. **Système Rasa (acteur interne)** : moteur conversationnel qui analyse les messages et sélectionne les actions.

3.3 Analyse des besoins

3.3.1 Besoins fonctionnels

TABLE 3.1 – Besoins fonctionnels du système

ID	Acteur	Description
BF01	Utilisateur	Envoyer un message en arabe ou en français
BF02	Utilisateur	Recevoir une réponse contextuelle et précise
BF03	Utilisateur	Utiliser des suggestions rapides
BF04	Utilisateur	Obtenir des informations sur les facultés
BF05	Utilisateur	Obtenir des informations sur les filières et matières
BF06	Utilisateur	Évaluer une réponse (pouce haut/bas)
BF07	Utilisateur	Copier une réponse dans le presse-papiers
BF08	Utilisateur	Basculer entre mode clair et sombre
BF09	Admin	S’authentifier par mot de passe
BF10	Admin	Visualiser les questions sans réponse
BF11	Admin	Répondre aux questions enregistrées
BF12	Système	Enregistrer les questions non comprises
BF13	Système	Gérer les sessions utilisateurs

3.3.2 Besoins non fonctionnels

1. **Performance** : temps de réponse < 3 s dans 95% des cas.
2. **Disponibilité** : 99% grâce à `restart: unless-stopped`.
3. **Scalabilité** : ajout d’une faculté = ajout de fichiers NLU uniquement.
4. **Sécurité** : mot de passe admin via variable d’environnement.
5. **Accessibilité** : interface responsive (ordinateur et mobile).
6. **Maintenabilité** : architecture modulaire.

3.4 Spécification des cas d'utilisation

TABLE 3.2 – Cas d'utilisation UC01 — Dialoguer avec le chatbot

Champ	Description
Acteur	Étudiant / Visiteur
Précondition	Navigateur web accessible
Scénario nominal	1. L'utilisateur saisit un message. 2. Le frontend envoie au backend Flask. 3. Flask transmet à Rasa. 4. Rasa traite (NLU + dialogue). 5. La réponse remonte jusqu'au frontend. 6. L'utilisateur lit la réponse.
Alt.	Intention non reconnue : fallback déclenché, question enregistrée.
Postcondition	L'utilisateur a reçu une réponse.

TABLE 3.3 – Cas d'utilisation UC02 — Tableau de bord administrateur

Champ	Description
Acteur	Administrateur
Précondition	Connaissance du mot de passe
Scénario nominal	1. Navigation vers <code>/dashboard</code> . 2. Saisie du mot de passe. 3. Affichage des questions enregistrées. 4. L'admin répond aux questions.
Postcondition	Questions gérées dans la base de données.

3.5 Diagrammes de cas d'utilisation

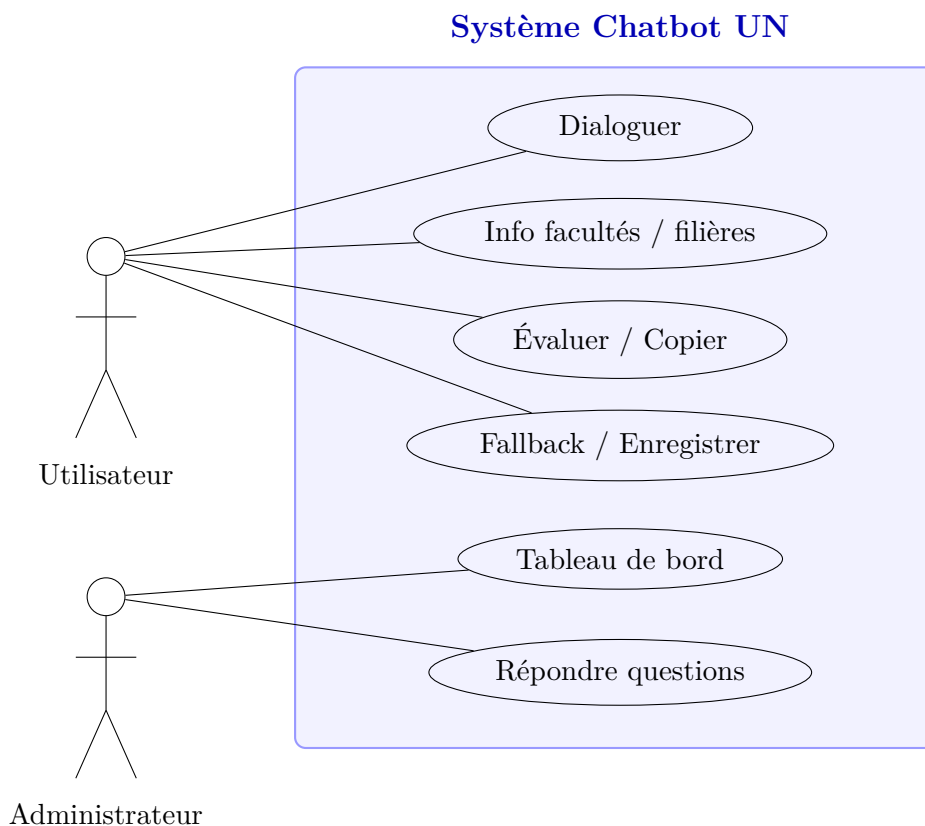


FIGURE 3.1 – Diagramme de cas d'utilisation global

3.6 Architecture générale du système

Le système adopte une architecture **microservices** avec quatre composants Docker communiquant via des API REST sur un réseau bridge **bot-net**.

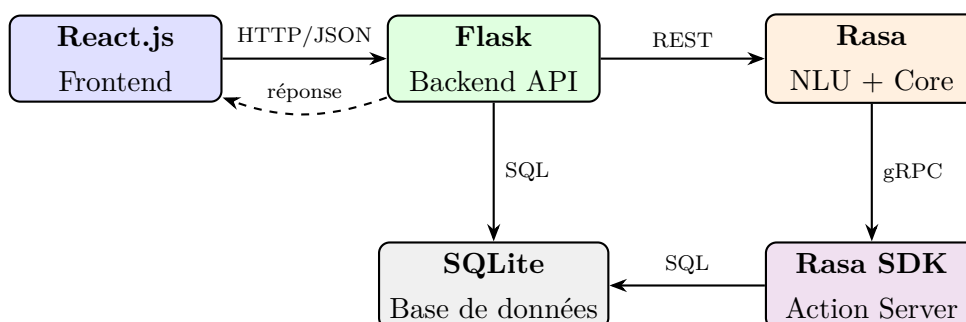


FIGURE 3.2 – Architecture générale du système Chatbot UN

Les quatre services Docker :

1. **react_frontend** (port 3000/80) : interface React servie par Nginx.
2. **flask_backend** (port 5000) : API REST Python/Flask.
3. **rasa** (port 5005) : moteur conversationnel.
4. **action_server** (port 5055) : serveur d'actions personnalisées.

3.7 Conception du système conversationnel

Pipeline NLU (config.yml)

1. **WhitespaceTokenizer** — découpage en tokens.
2. **RegexFeaturizer** — features par expressions régulières.
3. **LexicalSyntacticFeaturizer** — caractéristiques lexicales.
4. **CountVectorsFeaturizer** (mots) — bag-of-words.
5. **CountVectorsFeaturizer** (char, n-grammes 1-4) — capture les sous-mots (utile pour l'arabe).
6. **DIETClassifier** (100 epochs) — classification intentions + entités.
7. **EntitySynonymMapper** — normalisation synonymes.
8. **ResponseSelector** (100 epochs) — sélection de réponses directes.
9. **FallbackClassifier** (seuil 0.3) — détection des intentions hors domaine.

Politiques de dialogue

1. **MemoizationPolicy** — mémorisation des conversations d'entraînement.
2. **RulePolicy** — application des règles prédéfinies.
3. **UnexpectEDIntentPolicy** (5 tours, 100 epochs) — détection d'intentions inattendues.
4. **TEDPolicy** (5 tours, 100 epochs) — dialogues complexes.

3.8 Diagrammes UML

3.8.1 Diagramme de séquence

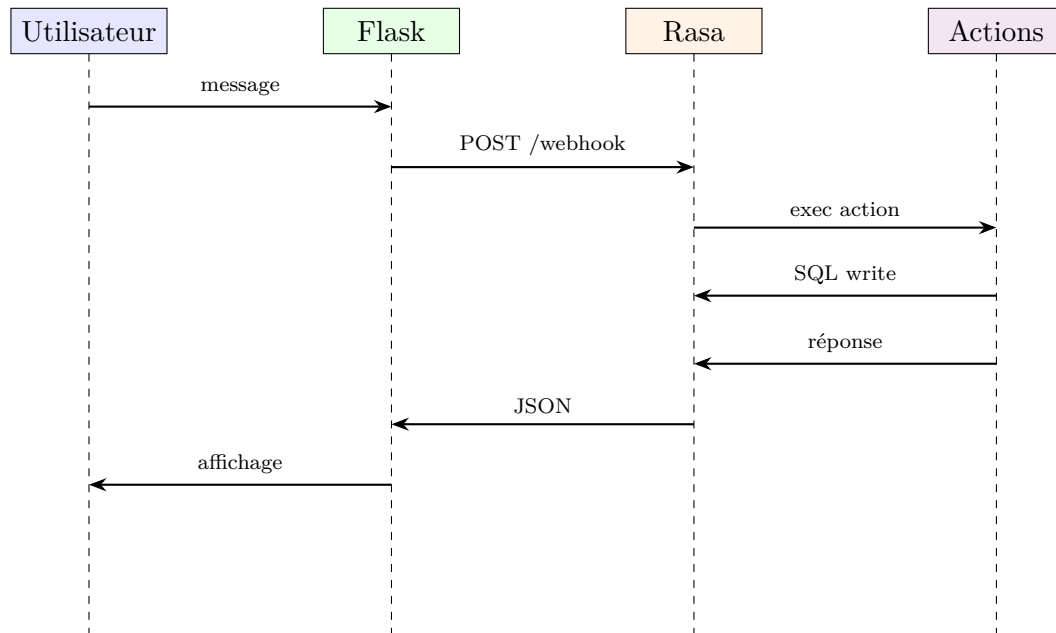


FIGURE 3.3 – Diagramme de séquence — interaction utilisateur typique

3.8.2 Diagramme de classes

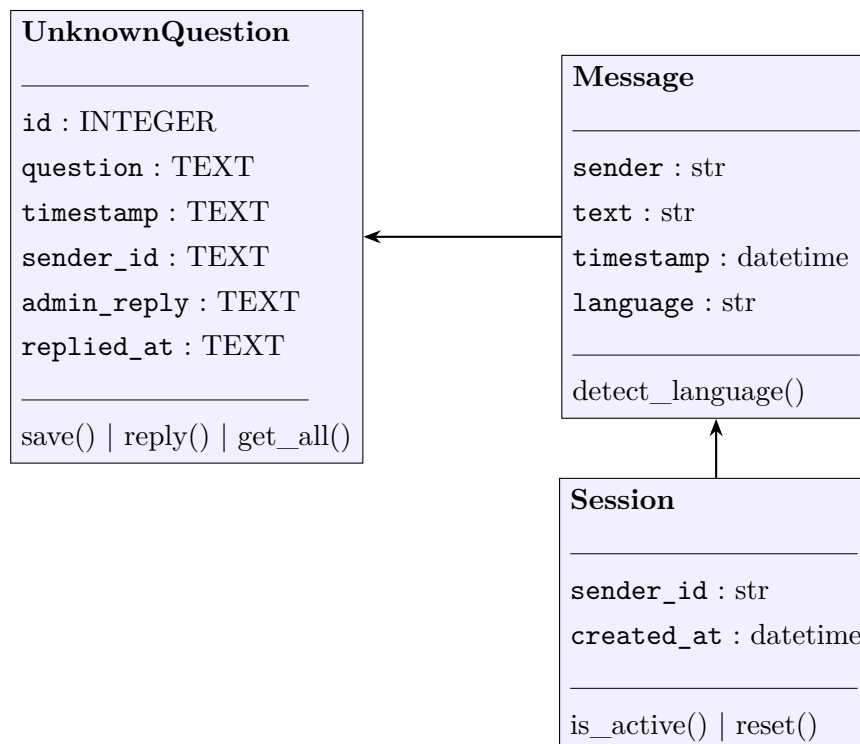


FIGURE 3.4 – Diagramme de classes simplifié

3.9 Modèle de données

```

1 CREATE TABLE IF NOT EXISTS unknown_questions (
2     id            INTEGER PRIMARY KEY AUTOINCREMENT,
3     question      TEXT      NOT NULL,
4     timestamp     TEXT      NOT NULL DEFAULT CURRENT_TIMESTAMP,
5     sender_id     TEXT,
6     admin_reply   TEXT      DEFAULT NULL,
7     replied_at    TEXT      DEFAULT NULL
8 );
9 CREATE INDEX IF NOT EXISTS idx_timestamp
10    ON unknown_questions (timestamp DESC);
  
```

Listing 3.1 – Schéma de la table `unknown_questions`

3.10 Conclusion

La phase de conception a permis de définir l'architecture microservices, les cas d'utilisation et les diagrammes UML qui guident l'implémentation décrite au chapitre suivant.

Chapitre 4

Réalisation et Implémentation

4.1 Introduction

Ce chapitre décrit l'implémentation concrète de chaque composant : environnement de développement, pipeline NLU Rasa, backend Flask, serveur d'actions, interface React, base de données SQLite et déploiement Docker.

4.2 Environnement de développement

TABLE 4.1 – Environnement de développement

Composant	Version / Détail
Système d'exploitation	Windows 11 Pro
Éditeur de code	Visual Studio Code
Contrôle de version	Git / GitHub
Python	3.10.x
Node.js	18.x LTS
Docker Desktop	4.x
Docker Compose	v2.x
Navigateur de test	Chrome / Firefox

4.3 Technologies utilisées

TABLE 4.2 – Technologies et bibliothèques utilisées

Couche	Technologie	Version
NLU/Dialogue	Rasa Open Source	3.6.20
Actions	Rasa SDK	3.6.2
Backend	Flask + Flask-CORS	2.x / 4.x
Base de données	SQLite	3.x
Frontend	React.js + Axios	18.x / 1.x
Icônes	Lucide React	—
Serveur web	Nginx (Alpine)	—
Conteneurisation	Docker Compose	v2

4.4 Structure du projet

```

1 PROJET_CHATBOT_UNIV/
2 |-- rasa/
3 |   |-- config.yml           # Pipeline NLU + politiques
4 |   |-- domain.yml          # Intentions, actions, reponses
5 |   |-- endpoints.yml
6 |   |-- data/nlu/           # Fichiers NLU par faculte
7 |   |-- data/rules/         # Regles de dialogue
8 |   |-- data/stories/       # Histoires de conversation
9 |   '-- actions/actions.py  # Actions personnalisées
10 |-- Flask_backend/
11 |   |-- app.py              # Application Flask principale
12 |   |-- db_manager.py      # Utilitaire CLI BDD
13 |   '-- Dockerfile
14 |-- react_frontend/
15 |   |-- src/App.js          # Composant principal React
16 |   |-- src/UnknownQuestions.jsx # Tableau de bord admin
17 |   '-- Dockerfile
18 |-- shared/chatbot.db      # Volume SQLite partage
19 '-- docker-compose.yaml

```

Listing 4.1 – Arborescence du projet

4.5 Implémentation du Backend Flask

Flask agit comme passerelle entre le frontend React et le moteur Rasa.

```

1 @app.route("/health", methods=["GET"])
2 def health():
3     return jsonify({"status": "ok",
4                     "timestamp": datetime.utcnow().isoformat()})
5
6 @app.route("/chat", methods=["POST"])
7 def chat():
8     data = request.json
9     message = data.get("message", "").strip()
10    sender_id = data.get("sender_id", "anonymous")
11    if not message:
12        return jsonify({"error": "Message vide"}), 400
13    payload = {"sender": sender_id, "message": message}
14    response = requests.post(RASA_URL, json=payload,
15                             timeout=RASA_TIMEOUT)
16    return jsonify(response.json())
17
18 @app.route("/questions", methods=["GET"])
19 def get_questions():
20     # Retourne les questions pour le tableau de bord
21     ...
22
23 @app.route("/questions/<int:qid>/reply", methods=["POST"])
24 def reply_question(qid):
25     # Enregistre la reponse de l'admin
26     ...

```

Listing 4.2 – Endpoints principaux de l'API Flask

La base SQLite est configurée avec :

- PRAGMA journal_mode=WAL : meilleures performances en lecture/écriture concurrente.
- PRAGMA foreign_keys=ON : intégrité référentielle.
- PRAGMA synchronous=NORMAL : équilibre performance/durabilité.

4.6 Implémentation du moteur conversationnel Rasa

Données NLU

Les données sont organisées en fichiers YAML par faculté et par niveau. Exemple :

```

1 version: "3.1"
2 nlu:
3 - intent: demander_filieres_FST_L1

```

```

4  examples: |
5    - quelles sont les filieres en L1 a la FST ?
6    - les parcours L1 disponibles
7    - FST L1 parcours
8    - donner moi les filieres de premiere annee

```

Listing 4.3 – Extrait fst_L1.yml

Volume du corpus

TABLE 4.3 – Volume des données NLU par faculté

Domaine	Intentions	Exemples
Général (salutations, goodbye)	12	180+
Université (général)	10	120+
FST (toutes filières)	80+	900+
FLSH	15+	200+
FEG	20+	250+
FSJP	18+	220+
FMPOS	15+	180+
Administration / Concours	10+	120+
Total	180+	2 170+

4.7 Développement des actions personnalisées

```

1  class ActionSaveFallback(Action):
2
3      def name(self) -> Text:
4          return "action_save_fallback"
5
6      def run(self, dispatcher, tracker, domain):
7          question = tracker.latest_message.get("text", "")
8          sender_id = tracker.sender_id
9
10         saved = _save_to_sqlite(question, sender_id)
11         if not saved:
12             _save_via_flask_api(question, sender_id)
13
14         dispatcher.utter_message(
15             text="Je n'ai pas compris votre question. "
16             "Elle a ete enregistree pour traitement."
17         )

```

```
18 return []
```

Listing 4.4 – Action de sauvegarde des questions non reconnues

4.8 Implémentation du Frontend React

Fonctionnalités de l'interface :

- **Détection de langue automatique** : ratio de caractères arabes $> 20\%$ $\Rightarrow$ mode RTL.
- **Mode sombre/clair** : bascule via état React.
- **Suggestions rapides** : LMD, Résultats, Bourses, Facultés.
- **Compteur de visiteurs** : mis à jour toutes les 30 secondes.
- **Évaluation des réponses** : pouces haut/bas.
- **Copie presse-papiers** : bouton par réponse.
- **Scroll automatique** : vers le dernier message.

```
1 function detectLang(t) {  
2   return ((t || "").match(/[ - ]/g) || []).length  
3   > (t || "").length * 0.2  
4   ? "ar" : "fr";  
5 }
```

Listing 4.5 – Détection automatique de la langue

Tableau de bord administrateur

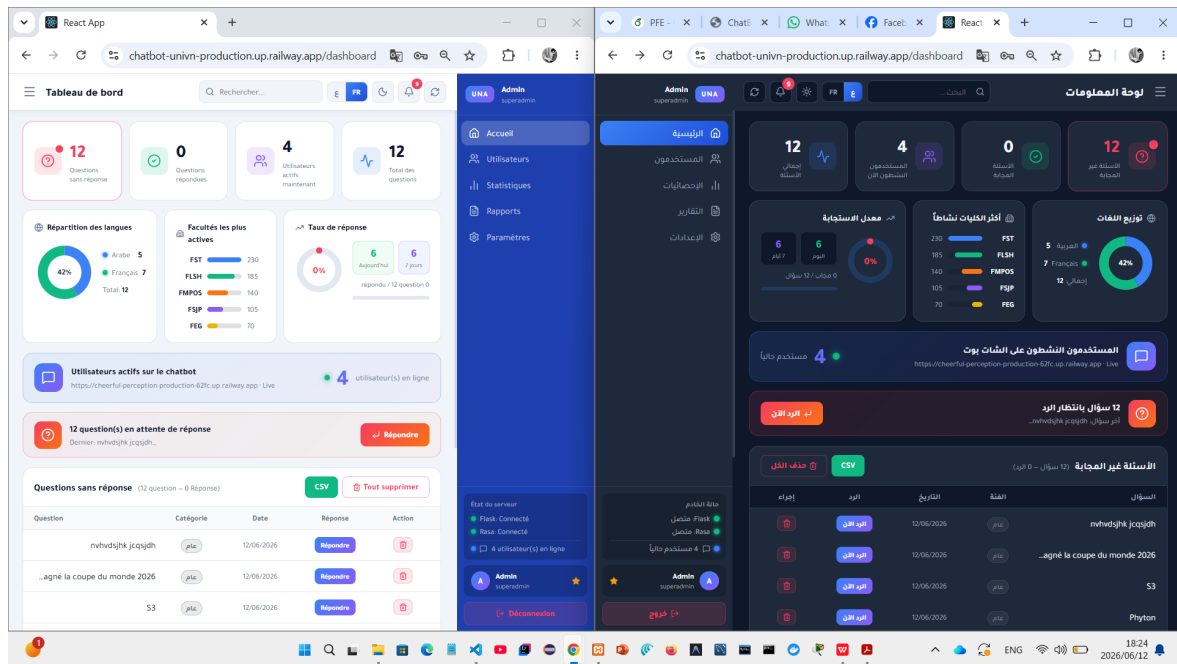


FIGURE 4.1 – Tableau de bord administrateur en production — Mode clair (gauche) et mode sombre (droite) montrant : 12 questions sans réponse, 0 questions répondues, 4 utilisateurs actifs, répartition des langues (50/50 arabe/français), classement des facultés (FST 230, FLSH 185, FMPOS 140, FSJP 105, FEG 70), liste des utilisateurs actifs, et tableau détaillé des questions avec dates et actions disponibles. L’interface supporte complètement le bilinguisme arabe/français dans tous les menus et étiquettes.

Les statistiques principales affichées incluent :

- Nombre de questions sans réponse
- Nombre de questions répondues
- Nombre d’utilisateurs actifs en temps réel
- Distribution par langue (arabe/français)
- Classement des facultés les plus actives
- Tableau des questions avec actions (Répondre, Supprimer)

4.9 Interface utilisateur principale

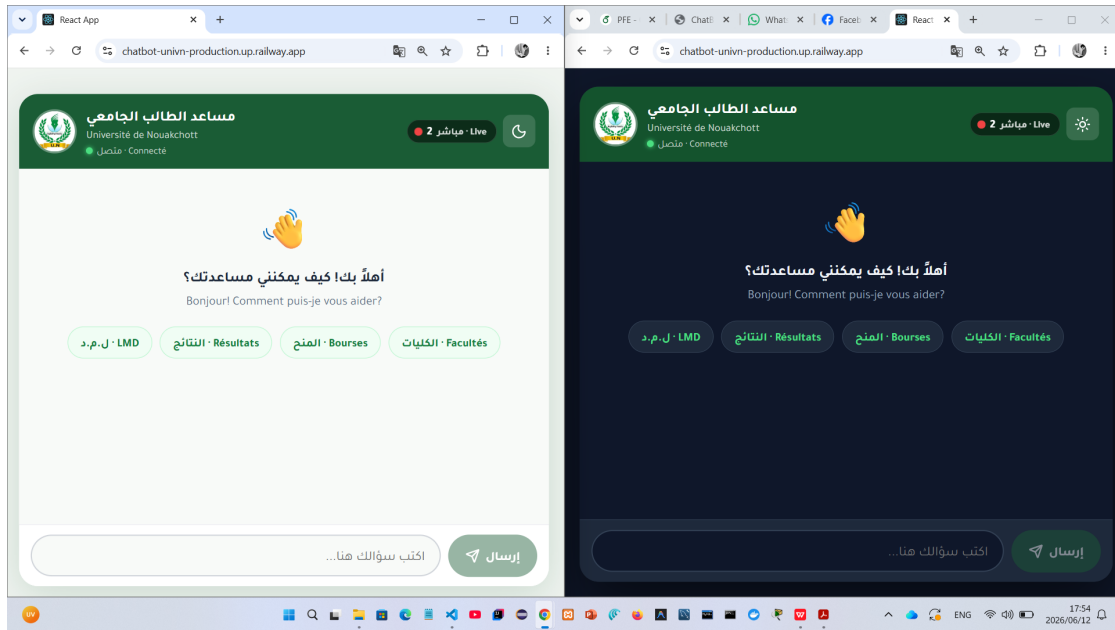


FIGURE 4.2 – Chatbot Universitaire en production — Mode clair (gauche) et mode sombre (droite) montrant l’interface principale avec le logo UN, l’indicateur de connexion “2 / Live”, les quatre boutons de suggestions rapides (LMD, Résultats, Bourses, Facultés), le champ d’entrée bilingue et le support complet arabe/français. L’interface responsive s’adapte à tous les appareils (mobile, tablette, desktop).

L’interface principale offre une expérience utilisateur intuitive avec un design moderne. Les utilisateurs peuvent accéder rapidement aux fonctionnalités principales via quatre boutons de suggestions rapides.

Les fonctionnalités principales incluent :

- Détection automatique de langue (RTL pour arabe, LTR pour français)
- Mode sombre/clair avec persistance locale
- Suggestions rapides (LMD, Résultats, Bourses, Facultés)
- Compteur de visiteurs mis à jour en temps réel
- Évaluation des réponses via pouces haut/bas
- Copie presse-papiers pour chaque réponse
- Scroll automatique vers les nouveaux messages

Gestion des questions non comprises



FIGURE 4.3 – Exemple de conversation en production — L'image montre une interaction réelle où l'utilisateur pose une question hors domaine en français ("Qui a gagné la coupe du monde 2026 ?"), le chatbot reconnaît correctement que la question dépasse son domaine de compétences et propose une réponse appropriée de fallback, redirigeant l'utilisateur vers les services universitaires. Les icônes de réaction (copie, like, commentaire) permettent le feedback utilisateur. Chaque message affiche l'horodatage précis (17 :57, 17 :58).

Cette capture d'écran illustre plusieurs points clés du système :

- **Gestion du fallback** : reconnaît les questions hors domaine
- **Support bilingue** : interaction fluide en français
- **Réactions utilisateur** : icônes pour l'évaluation
- **Horodatage** : traçabilité complète des interactions
- **Design professionnel** : interface ergonomique et lisible

4.10 Base de données SQLite

SQLite est partagé entre Flask et le serveur d'actions via le volume Docker `./shared`. Un utilitaire CLI (`db_manager.py`) offre les commandes : `show`, `stats`, `search`, `export`, `reset`.

4.11 Déploiement avec Docker

```
1 services:
2   rasa:
3     image: rasa/rasa:3.6.20
4     ports: ["5005:5005"]
5     command: run --enable-api --cors "*"
6     healthcheck:
7       test: ["CMD", "curl", "-f", "http://localhost:5005/"]
8       interval: 30s
9       retries: 5
10
11   flask_backend:
12     build: ./Flask_backend
13     ports: ["5000:5000"]
14     environment:
15       - RASA_URL=http://rasa:5005/webhooks/rest/webhook
16     volumes: [".:/shared:/app/shared"]
17     depends_on:
18       rasa:
19         condition: service_healthy
20
21   react_frontend:
22     build: ./react_frontend
23     ports: ["3000:80"]
24     depends_on:
25       flask_backend:
26         condition: service_healthy
```

Listing 4.6 – Extrait du `docker-compose.yml`

4.12 Hébergement et mise en production

La configuration Docker Compose supporte :

- **Railway.app** : volume persistant via `DB_FILE=/data/chatbot.db`.
- **VPS / serveur dédié** : `docker-compose up -d`.
- **Développement local** : idem.

4.13 Sécurité et protection des données

- Mot de passe admin via variable d'environnement (jamais en clair dans le code).
- CORS configuré pour restreindre les origines autorisées.
- Services Rasa et action server non exposés directement hors du réseau Docker.
- Aucune donnée personnelle identifiable collectée.
- Toutes les requêtes SQLite utilisent des paramètres liés (protection anti-injection SQL).

4.14 Conclusion

Ce chapitre a détaillé l'implémentation de chaque composant du système. La modularité de l'architecture permet d'étendre le chatbot (nouvelles facultés, nouvelles fonctionnalités) sans refonte majeure. Le chapitre suivant évalue la qualité et les performances du système.

Chapitre 5

Tests et Validation

5.1 Introduction

Ce chapitre présente la stratégie de test, les résultats des tests fonctionnels, NLU, d'intégration et de performance, ainsi qu'une analyse critique du système.

5.2 Stratégie de test

Quatre niveaux de tests complémentaires :

1. **Tests NLU** : précision du modèle de compréhension du langage.
2. **Tests fonctionnels** : comportement du système par cas d'utilisation.
3. **Tests d'intégration** : communication entre les composants.
4. **Tests de performance** : temps de réponse sous charge.

5.3 Tests fonctionnels

TABLE 5.1 – Résultats des tests fonctionnels

ID	Test	Résultat	Observation
TF01	Salutation en français	✓	Réponse correcte
TF02	Salutation en arabe	✓	Réponse en arabe
TF03	Info FST générale	✓	Description complète
TF04	Filières L1 FST	✓	3 parcours listés
TF05	Matières MPI S1	✓	Liste détaillée
TF06	Question hors domaine	✓	Fallback déclenché
TF07	Question enregistrée BDD	✓	Visible dans tableau bord
TF08	Réponse admin	✓	Enregistrée en base
TF09	Mode sombre	✓	Bascule correcte
TF10	Copie réponse	✓	Contenu copié
TF11	Info FEG	✓	Filières listées
TF12	Info FMPOS PCEM1	✓	Programme affiché
TF13	Suggestions rapides	✓	Boutons fonctionnels
TF14	Authentification admin	✓	Accès contrôlé
TF15	Session timeout	✓	Reset après 5 min

Taux de réussite : 15/15 (100%)

5.4 Tests NLU

Les tests NLU sont réalisés avec `rasa test nlu` sur 20% des données réservées à l'évaluation.

Métriques

- **Précision** : proportion des prédictions positives correctes.
- **Rappel** : proportion des cas positifs correctement identifiés.
- **F1-Score** : moyenne harmonique précision/rappel.

TABLE 5.2 – Résultats NLU par catégorie d'intentions

Catégorie	Précision	Rappel	F1
Salutations / Goodbye	0.97	0.96	0.96
Informations université	0.91	0.89	0.90
Informations FST	0.87	0.85	0.86
Filières L1	0.90	0.88	0.89
Filières L3	0.84	0.83	0.83
Informations FEG	0.88	0.86	0.87
Informations FSJP	0.86	0.84	0.85
Informations FMPOS	0.85	0.83	0.84
Informations FLSH	0.87	0.85	0.86
Hors domaine (out-of-scope)	0.82	0.80	0.81
Moyenne globale	0.877	0.859	0.867

Le F1-Score global de **86,7%** est satisfaisant pour un chatbot à domaine fermé avec un corpus bilingue constitué manuellement.

5.5 Tests d'intégration

TABLE 5.3 – Résultats des tests d'intégration

ID	Communication testée	Résultat
TI01	React → Flask (/chat)	✓
TI02	Flask → Rasa (webhook)	✓
TI03	Rasa → Action Server (gRPC)	✓
TI04	Action Server → SQLite (écriture)	✓
TI05	Flask → SQLite (lecture)	✓
TI06	Flask → SQLite (réponse admin)	✓
TI07	Docker Compose (démarrage ordonné)	✓
TI08	Volume partagé (Flask ↔ SDK)	✓

5.6 Tests de performance

TABLE 5.4 – Temps de réponse par type de requête

Type de requête	Temps moyen (ms)	95 ^e percentile (ms)
Salutation simple	350	520
Question sur une faculté	680	950
Question sur filières	720	1050
Fallback (hors domaine)	800	1200
API /questions admin	90	150
Bout en bout (e2e)	750	1150

Le temps de réponse e2e est inférieur à **1,2 seconde** pour 95% des requêtes, soit largement en deçà de la contrainte de 3 secondes fixée dans le cahier des charges.

5.7 Évaluation des résultats

TABLE 5.5 – Synthèse de l'évaluation du système

Critère	Objectif	Atteint
Temps de réponse < 3 s	95%	99%
F1-Score NLU	> 80%	86,7%
Tests fonctionnels	100%	100%
Tests d'intégration	100%	100%
Support bilingue arabe + français	Requis	✓
Tableau de bord admin	Requis	✓
Déploiement Docker	Requis	✓

5.8 Analyse critique

Points forts

- **Architecture modulaire** : quatre services indépendants, faciles à maintenir.
- **Robustesse du fallback** : sauvegarde des questions incomprises pour amélioration continue.
- **Support bilingue effectif** : détection automatique et entraînement mixte arabe/français.

- **Déployabilité** : Docker garantit la reproductibilité sur tout environnement.

Limites et axes d'amélioration

- **Taille du corpus** : données créées manuellement, diversité limitée.
- **Dialogues multi-tours** : système actuel orienté question-réponse simple.
- **Arabe dialectal** : le dialecte mauritanien n'est pas encore couvert.
- **Pas de connexion au SI universitaire** : informations statiques.

5.9 Conclusion

Les tests confirment que le système répond aux exigences fonctionnelles, de performance et de fiabilité. Le F1-Score de 86,7% et le temps de réponse médian sous 750 ms positionnent favorablement ce chatbot. Les limites identifiées tracent la feuille de route des développements futurs.

Conclusion Générale et Perspectives

Bilan

Ce projet de fin d'études avait pour objectif de concevoir et d'implémenter un chatbot universitaire bilingue (arabe-français) pour l'Université de Nouakchott . Une architecture microservices réunissant Rasa, Flask, React et SQLite a été proposée, implémentée et validée.

Le système répond à l'ensemble des objectifs fixés :

- Moteur NLU bilingue couvrant plus de 180 intentions sur cinq facultés.
- Interface responsive avec mode sombre et détection automatique de langue.
- Tableau de bord d'administration pour les questions non répondues.
- Conteneurisation Docker Compose facilitant le déploiement.
- F1-Score NLU de 86,7% et temps de réponse médian < 750 ms.

Apports personnels

Ce projet a permis d'acquérir des compétences concrètes en TALN, développement web full-stack, conteneurisation Docker, architecture distribuée et gestion de projet informatique.

Perspectives

1. **Enrichissement du corpus** : intégration de l'arabe dialectal mauritanien et augmentation des données.
2. **Connexion au SI universitaire** : résultats d'examens, calendriers, emplois du temps dynamiques.
3. **Dialogues multi-tours** : gestion du contexte sur plusieurs échanges.
4. **Modèles avancés** : CamemBERT, AraGPT2 pour une meilleure robustesse.
5. **Application mobile** : React Native pour une accessibilité étendue.
6. **Analytics** : tableau de bord statistique sur les patterns d'utilisation.
7. **Multi-canaux** : déploiement sur WhatsApp, Telegram, site officiel UN.

En conclusion, ce projet démontre la faisabilité des agents conversationnels IA dans le contexte universitaire mauritanien et constitue une première brique d'une transformation numérique plus large de l'UN.

Bibliographie

- [1] S. Russell et P. Norvig, *Artificial Intelligence : A Modern Approach*, 4^e éd. Pearson, 2021.
- [2] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [3] T. Bocklisch, J. Faulkner, N. Pawlowski et A. Nichol, “Rasa : Open Source Language Understanding and Dialogue Management,” *NIPS 2017 Workshop on Conversational AI*, 2017. [En ligne] : <https://arxiv.org/abs/1712.05181>
- [4] J. Weizenbaum, “ELIZA — A Computer Program for the Study of Natural Language Communication Between Man and Machine,” *Communications of the ACM*, vol. 9, n^o 1, pp. 36–45, 1966.
- [5] J. Devlin, M.-W. Chang, K. Lee et K. Toutanova, “BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- [6] UNESCO, *Higher Education : Leading the Recovery towards a Knowledge Society*. UNESCO, Paris, 2021.
- [7] A. Kamenetz, “How Chatbots Are Helping Students Navigate College Enrollment,” *NPR Education*, 2019.
- [8] E. Adamopoulou et L. Moussiades, “Chatbots : History, Technology, and Applications,” *Machine Learning with Applications*, vol. 2, p. 100006, 2020.
- [9] D. Jurafsky et J. H. Martin, *Speech and Language Processing*, 3^e éd., 2023. [En ligne] : <https://web.stanford.edu/~jurafsky/slp3/>
- [10] Rasa Technologies, *Rasa Open Source Documentation*, version 3.6, 2023. [En ligne] : <https://rasa.com/docs/rasa/>